Computer Organization and Architecture
(Code: CS256)

Lecture-18: ALU Operations II

Course Faculty

Dr. B. R. Bhowmik

Assistant Professor

Dept. of CSE

Website: https://sites.google.com/view/biswajitrbhowmik
https://sites.google.com/view/vicayana-gaveshana

Course Email: sangsangstha@gmail.com

Arithmetic Multiplication

- *The numbers that are multiplied are called the* ***factors.***
- The result that is obtained after the multiplication of two or more numbers is known as the **product** of those numbers.
- The number to be multiplied with the given number is called a **multiplicand.**
- The number which multiplies the given multiplicand is called a **multiplier.**

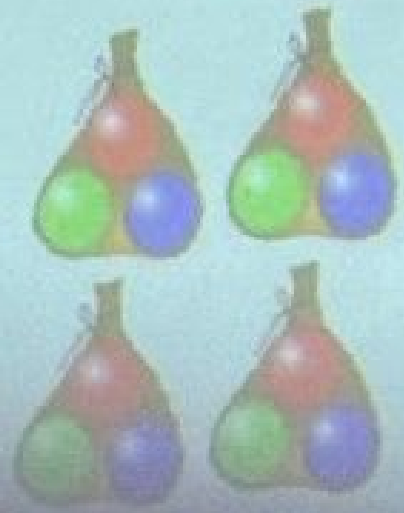
factor factor

4 x 5 = 20

product

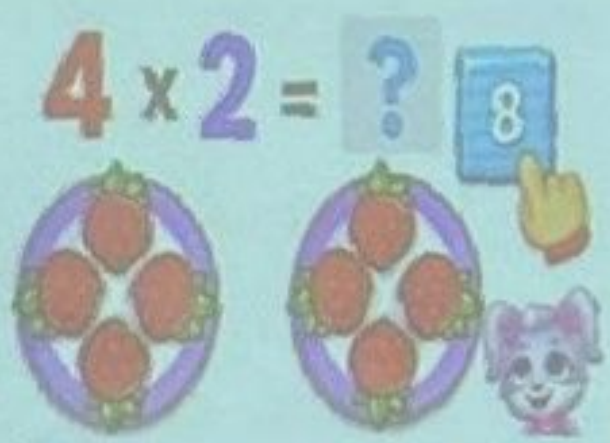
2 x 9 = 18

Multiplicand Multiplier Product



Arithmetic Multiplication

- *Multiplication is a method of finding the product of two or more numbers.*
- It is one of the basic arithmetic operations, that we use in everyday life.
- *Multiplication is an operation that represents the basic idea of repeated addition of the same number.*
- Multiplication can be performed as a sequence of repeated additions.
- *A* * *B* or AxB is interpreted as add *A*, *B* times.
- A+A+A+..... B times



Application of Multiplication

- Multiplication is used when we need to combine groups of equal sizes.
- Modern processors perform many multiplications & divisions:
  - Encryption,
  - Image compression,
  - Graphic rendering

Multiplication Approach

- *Sequential Multiplication or Add-Shift Algorithm*
  - Method-1
  - Method-2
  - Method-3
- **Booth's Algorithm**
- Combinational Multiplication
- MIPS Multiplication Instructions

Shift-Add Multiplication

- Basic algorithm analogous to decimal multiplication
- Break multiplier into digits
- Multiply one digit at a time;
  shift multiplicand to form partial products
- Create product as sum of partial products

```
Multiplicand        0110      (6)
Multiplier        x 0011      (3)
                  --------------
                    0110
                   0110
                  0000
                 0000
                --------------
Product         00010010    (18)
```

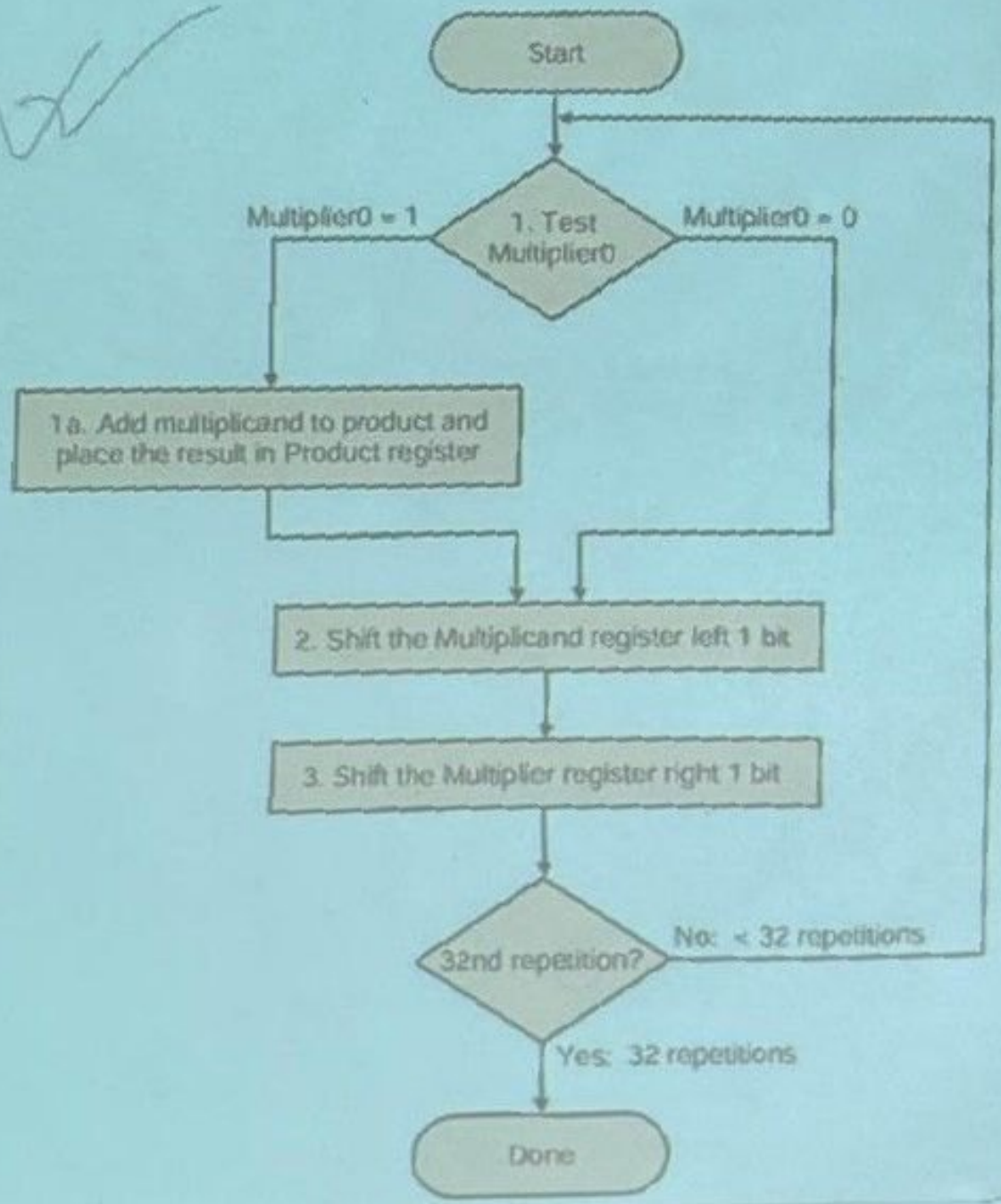
Partial Products

Sequential Multiplier - First Version

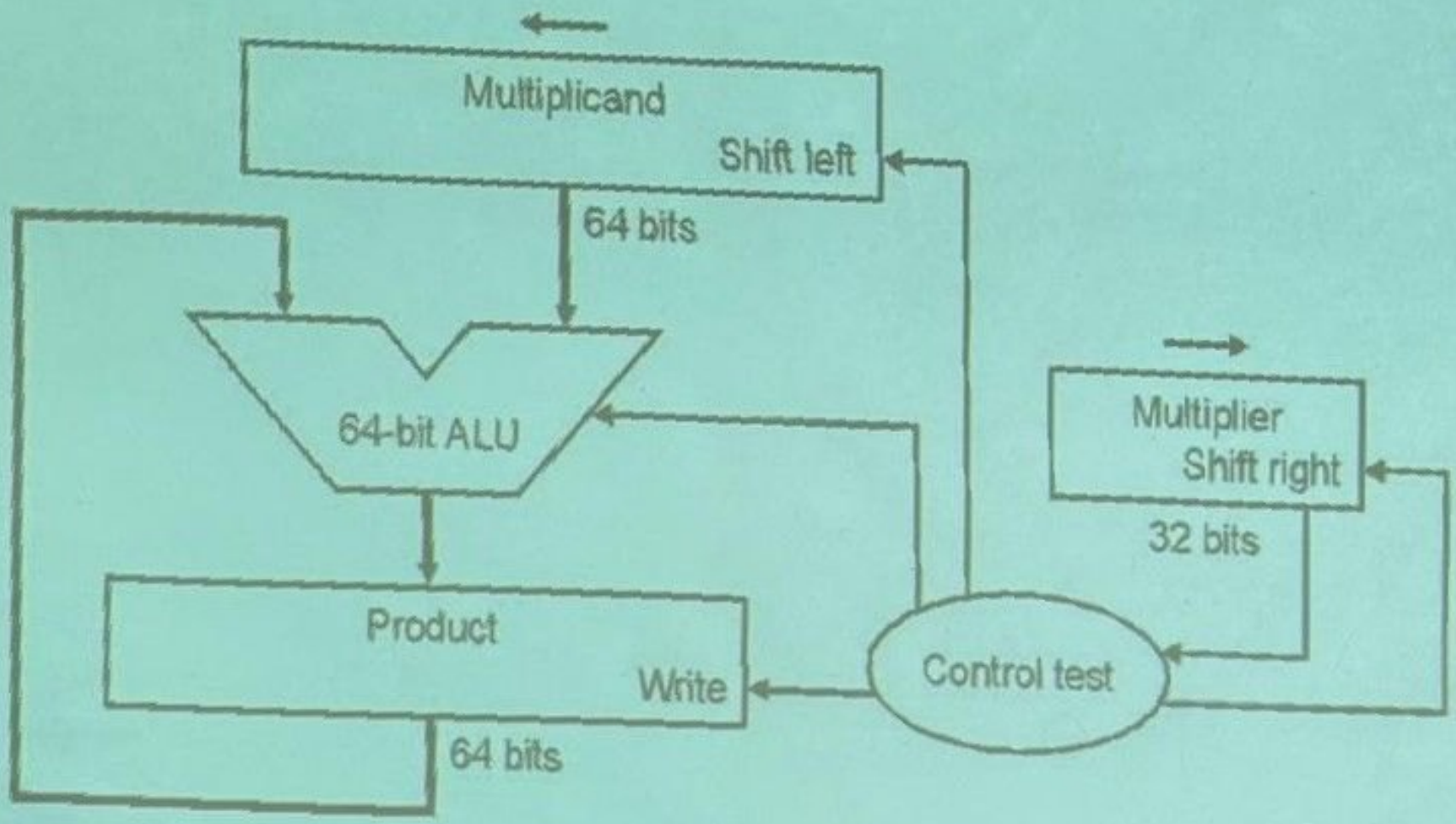
- Multiplicand shifts *left*
- Multiplier shifts *right*
- Sample LSB of multiplier to decide whether to *add*
- n bit multiplicand X m bit multiplier = *(n+m-1)* bit product

Sequential Multiplier - First Version

```
        1 0 0 0
      x 1 0 0 1
      ---------
        1 0 0 0
      0 0 0 0
    0 0 0 0
  1 0 0 0
  -------------
  1 0 0 1 0 0 0
```



Sequential Multiplier - First Version

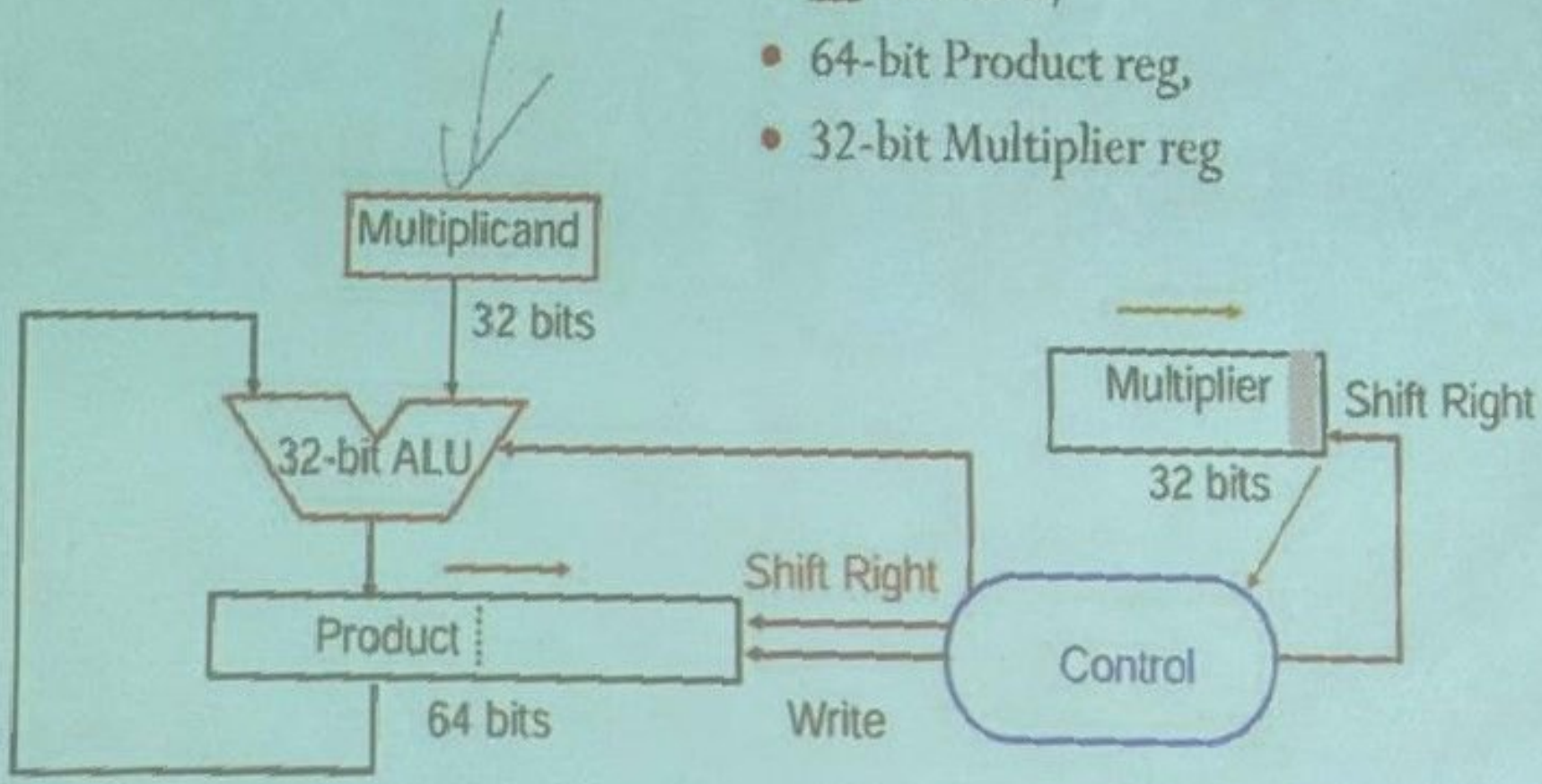


Multiplier Limitations and Improvements

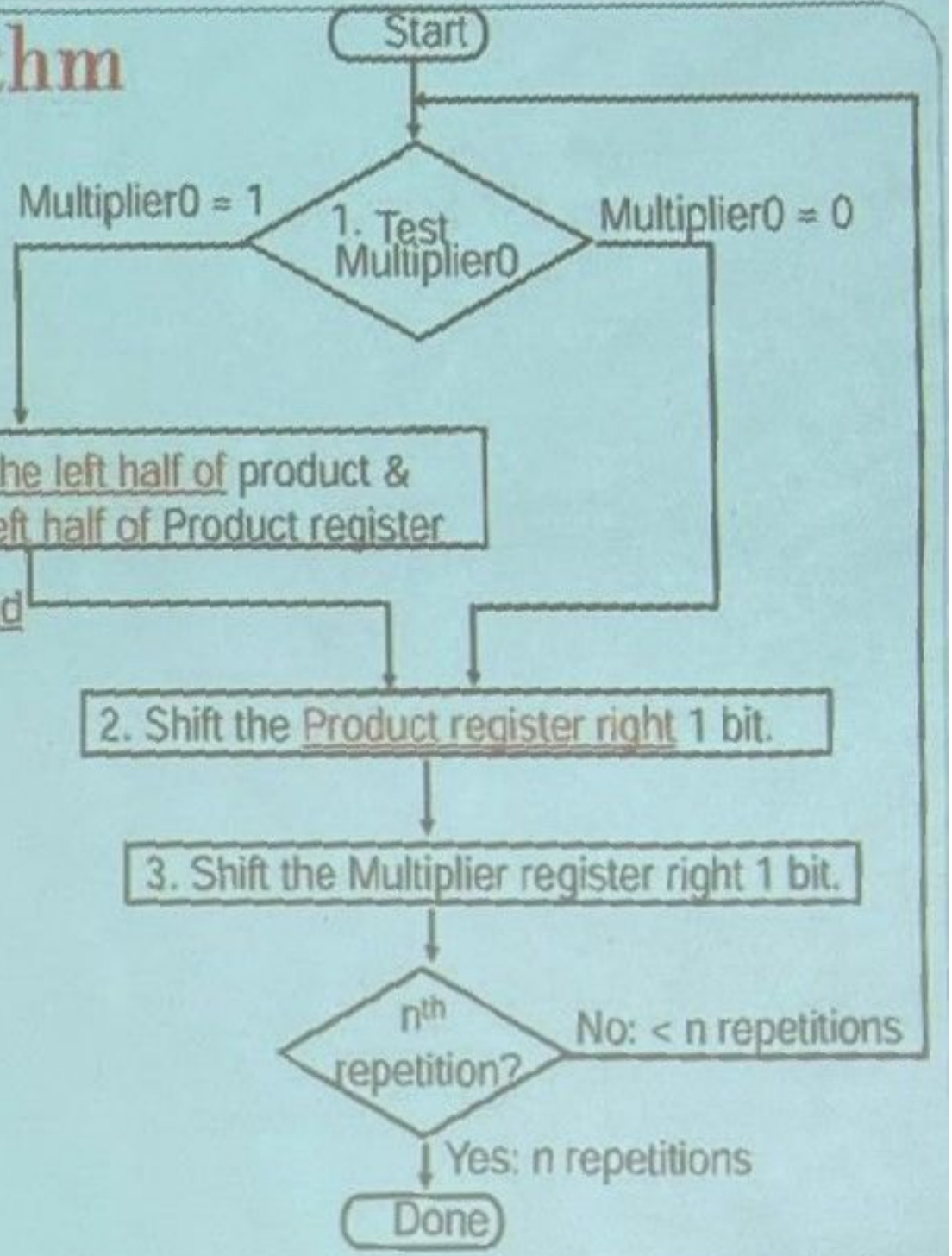
- Do we really need a 64-bit adder?
  - No, since low-order bits are not involved
  - 1/2 (50%) bits in multiplicand always 0
    => 64-bit adder is wasted
  - 0's inserted in left of multiplicand as shifted
    => least significant bits of product never changed once formed
  - Hence, just use a 32-bit adder
    - Shift product register right on every step Instead of shifting multiplicand to left.

Multiply Hardware Version 2

- 32-bit Multiplicand reg,
- 32 -bit ALU,
- 64-bit Product reg,
- 32-bit Multiplier reg



Multiply Algorithm Version 2



	Product	Multiplier	Multiplicand
	0000 0000	0011	0010
1:	0010 0000	0011	0010
2:	0001 0000	0011	0010
3:	0001 0000	0001	0010
1:	0011 0000	0001	0010
2:	0001 1000	0001	0010
3:	0001 1000	0000	0010
1:	0001 1000	0000	0010
2:	0000 1100	0000	0010
3:	0000 1100	0000	0010
1:	0000 1100	0000	0010
2:	0000 0110	0000	0010
3:	0000 0110	0000	0010
	0000 0110	0000	0010

Limitations of Multiply Version 2

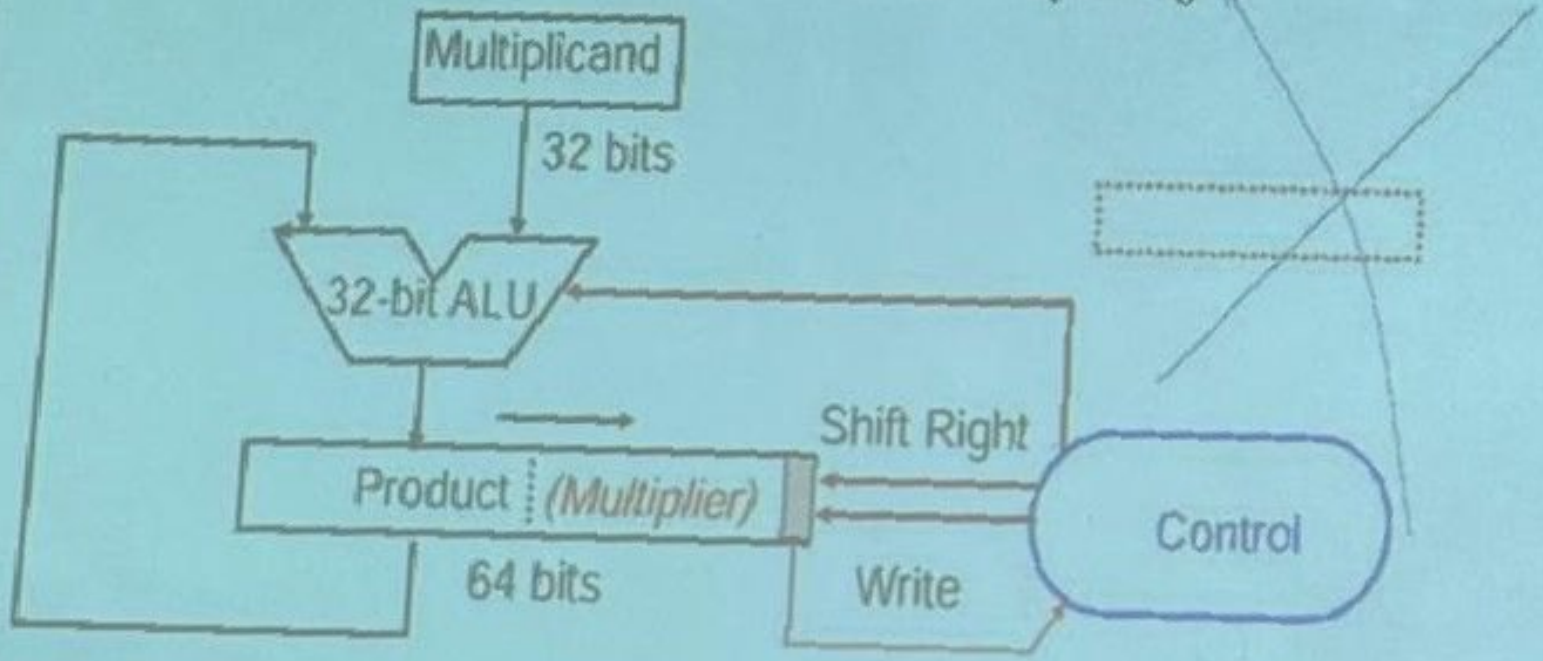
- Product register wastes space that exactly matches size of multiplier
- Both Multiplier register and Product register require right shift
- Combine Multiplier register and Product register

Multiply Algorithm Version 2 Illustration

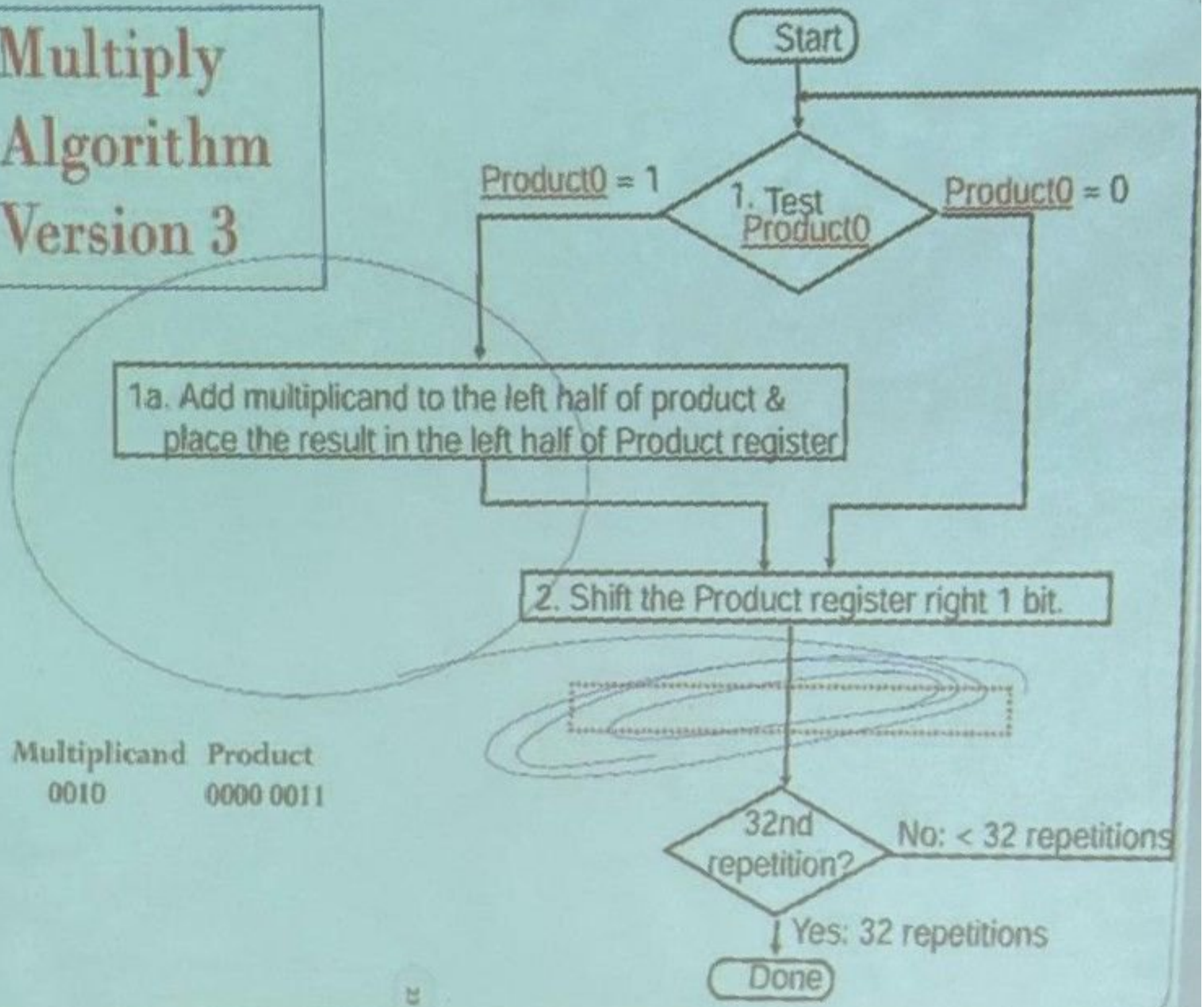
M'ier: 0011	**Mcand: 0010**	**P: 0000 0000**	
1a. 1=>P=P+Mcand	M'ier: 0011	Mcand: 0010	P: 0010 0000
2. Shr P	M'ier: 0011	Mcand: 0010	P: 0001 0000
3. Shr M'ier	M'ier: 0001	Mcand: 0010	P: 0001 0000
1a. 1=>P=P+Mcand	M'ier: 0001	Mcand: 0010	P: 0011 0000
2. Shr P	M'ier: 0001	Mcand: 0010	P: 0001 1000
3. Shr M'ier	M'ier: 0000	Mcand: 0010	P: 0001 1000
1. 0=>nop	M'ier: 0000	Mcand: 0010	P: 0001 1000
2. Shr P	M'ier: 0000	Mcand: 0010	P: 0000 1100
3. Shr M'ier	M'ier: 0000	Mcand: 0010	P: 0000 1100
1. 0=>nop	M'ier: 0000	Mcand: 0010	P: 0000 1100
2. Shr P	M'ier: 0000	Mcand: 0010	P: 0000 0110
3. Shr M'ier	M'ier: 0000	Mcand: 0010	P: 0000 0110

Multiply Hardware Version 3

- 32-bit Multiplicand reg,
- 32 -bit ALU,
- 64-bit Product reg,
- *0-bit Multiplier reg*



Multiply Algorithm Version 3



Multiplicand	Product
0010	0000 0011

Multiply Algorithm Version 2 Illustration

Mcand: 0010 P: 0000 0000

1a. 1 => P = P + Mcand Mier: 0011 Mcand: 0010 P: 0010 0000 2.

Shr P Mier: 0011 Mcand: 0010 P: 0001 0000

3. Shr Mier Mier: 0001 Mcand: 0010 P: 0001 0000

1a. 1 => P = P + Mcand Mier: 0001 Mcand: 0010 P: 0011 0000 2.

Shr P Mier: 0001 Mcand: 0010 P: 0001 1000

3. Shr Mier Mier: 0000 Mcand: 0010 P: 0001 1000

1. 0 => nop Mier: 0000 Mcand: 0010 P: 0001 1000

2. Shr P Mier: 0000 Mcand: 0010 P: 0000 1100

3. Shr Mier Mier: 0000 Mcand: 0010 P: 0000 1100

1. 0 => nop Mier: 0000 Mcand: 0010 P: 0000 1100

2. Shr P Mier: 0000 Mcand: 0010 P: 0000 0110

3. Shr Mier Mier: 0000 Mcand: 0010 P: 0000 0110

Observations on Multiply Version 3

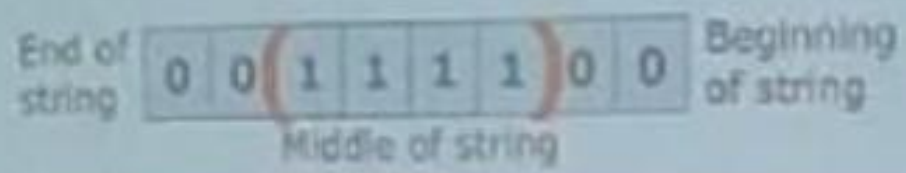
- 2 steps per bit because Multiplier & Product combined
- MIPS registers High and Low are left and right half of Product
- Gives us MIPS instruction MultU
- *What about signed multiplication?*
  - easiest solution is to make both positive & remember whether to complement product when done (leave out the sign bit, run for 31 steps)
  - apply definition of 2's complement
    - need to sign-extend partial products and subtract at the end
  - **Booth's Algorithm is elegant** way to multiply signed numbers using same hardware as before and save cycles
    - can be modified to handle multiple bits at a time

Signed Multiplication

- Previous algorithms work for unsigned.
- We could:
  - convert both multiplier and multiplicand to positive before starting, but remember the signs.
  - adjust the product to have the right sign (might need to negate the product).
  - *Set the sign bit to make the number negative if the multiplicand and multiplier disagree in sign*
    - Positive × Positive = Positive
    - Negative × Negative = Positive
    - Positive × Negative = Negative
    - Negative × Positive = Negative

Booth's Approach Explanation

- Booth also works for negative numbers
- Booth Algorithm
  - Add additional bit to the right of product
  - Use right two bits of product to determine action
  - Use arithmetic right shift



Current Bit	Right Bit	Explanation	Action
0	0	Middle of string of 0s	Do nothing
0	1	End of string of 1s	Add multiplicand
1	0	Beginning of string of 1s	Subtract multiplicand
1	1	Middle of string of 1s	Do nothing

Booth's Multiplication

Booth's Multiplication

- The algorithm was invented by *Andrew Donald Booth* in 1950 while doing research on crystallography
- Originally proposed to reduce addition steps
- *Booth's Algorithm is an elegant way to multiply signed numbers using same hardware as before and save cycles*
- Can handle multiple bits at a time
- Remember signs and do unsigned multiplication.
- *Bonus: works for two's complement numbers*
- Uses shifting, addition, and subtraction

Motivation for Booth's Algorithm

- Example 2 x 6 = 0010 x 0110:

```
                0010
x               0110
+               0000      shift (0 in multiplier)
+              0010       add (1 in multiplier)
+             0010        add (1 in multiplier)
+            0000         shift (0 in multiplier)
            00001100
```

- ALU with add or subtract gets same result in more than one way:

```
6      = - 2 + 8
0110   = - 00010 + 01000
```

- For example

```
              0010
x             0110
              0000  shift (0 in multiplier)
-           0010    sub (start string of 1's)
            0000    shift (mid string of 1's)
+         0010      add (end string of 1's)
        00001100
```

Booth's Approach

Assume that

1. Let M is the multiplicand.
2. Let Q is the multiplier.
3. Consider a 1-bit register Q_{-1} and initialize it to 0.
4. Consider a register A and initialize it to 0.

Procedure

1. If $Q_0 Q_{-1}$ are same i.e. 00 or 11 then, perform arithmetic right shift by 1 bit.
2. If $Q_0 Q_{-1}$ = 10 then perform

   A←A-M

   And then perform arithmetic right shift.
3. If $Q_0 Q_{-1}$ = 01 then perform

   A←A+M

   And then perform arithmetic right shift.

Booth's Approach Example

INITIALISE	A	Q	Q_{-1}	OPERATIONS
	0 0 0 0 ↓ ↘ ↘ ↘ ↘	0 0 1 0 ↘ ↘ ↘ ↘	0	
Step 1.	0 0 0 0	0 0 0 1 ↓	0 ↓	Arithmetic right shift
Step 2.	1 0 1 0 ↓ ↘ ↘ ↘ ↘ 1 1 0 1	0 0 0 1 ↘ ↘ ↘ ↘ 0 0 0 0	0 1	A←A-M Then shift
Step 3.	0 0 1 1 ↓ ↘ ↘ ↘ ↘ 0 0 0 1 ↓ ↘ ↘ ↘ ↘	0 0 0 0 ↘ ↘ ↘ ↘ 1 0 0 0 ↘ ↘ ↘ ↘	1 0	A←A+M Then shift
Step 4.	0 0 0 0	1 1 0 0 In binary, 12 = 1100 Hence 3*2 = 12	0 0	Arithmetic right shift

Booth's Approach Example 2

A	Q	Q-1	M		
0000	0101	0	0111	Initial value	
1001	0101	0	0111	A ← A-M	First cycle
1100	1010	1	0111	shift	
0011	1010	1	0111	A ← A+M	Second cycle
0001	1101	0	0111	shift	
1010	1101	0	0111	A ← A-M	Third cycle
1101	0110	1	0111	shift	
0100	0110	1	0111	A ← A+M	Fourth cycle
0010	0011	0	0111	shift	

Booth's Approach Example

- Multiply 6 and 2 using the Booth's Multiplication Approach
- Here M = 6 = 0110
- Q = 2 = 0010

Booth's Multiplication

- In each iteration two bits of multiplier are being investigated and
- Proper action(s) are taken according to the following coding table:

00	no action	shift right once
01	add multiplicand	shift right once
10	sub multiplicand	shift right once
11	no action	shift right once

Booth's Algorithm

- Observation: if we can both add and subtract, there are multiple ways to create a product
- Example: multiply 2_{ten} by 6_{ten} (0010_{two} X 0110_{two})
  - Product = (2 X 2) + (2 X 4) OR
  - Product = (2 X -2) + (2 X 8)

Regular Algorithm

```
        0010
X       0110
+       0000   shift
+      0010    shift + add
+     0010     shift + add
+    0000      shift
    00001100
```

Booth's Algorithm

```
        0010
X       0110
        0000   shift
-      0010    shift + subtract
      0000     shift
+    0010      shift + add
    00001100
```

An overview of Booth's Algorithm

- Booth's algorithm is an extension to the add-and-shift approach.
- When doing multiplication, strings of 0s in the multiplier require only shifting (no addition).
- When doing multiplication, strings of 1s in the multiplier require operation and shifting.
- We need to add or subtract only at positions in the multiplier where ther is a transition from a 0 to a 1, or from a 1 to a 0.

Modified Booth's Algorithm

- Check 3 bits of multiplier at a time and take proper steps as follows:

000	no action	shift right twice
001	add multiplicand	shift right twice
010	add multiplicand	shift right twice
011	add 2*multiplicand	shift right twice
100	sub 2*multiplicand	shift right twice
101	sub multiplicand	shift right twice
110	sub multiplicand	shift right twice
111	no action	shift right twice

Modified Booth's Algorithm Example

001011 * 011001

Extension

000000	0110010	
001011		010 ⇒ add B, shift twice
001011	0110010	
000010	1101100	
101010		100 ⇒ sub 2B, shift twice
101100	1101100	
111011	0011011	
010110		011 ⇒ add 2B, shift twice
010001	0011011	
0001100	0100011 0	

Answer

Observations on Booth's Algorithm

- Any version of Booth's algorithm allows a sequence of consecutive 1^s to be bypassed.
- Modified Booth's Algorithm is faster than Booth's Algorithm.
- Booth's Algorithm can be further extended by looking at 4 bits, (5 bits, ...) at a time and taking proper actions according to the proper encoding table.

A7